9
Demystifying the Impact of AI

Everything that has a beginning has an end. We have reached the final chapter in our journey together.

You haven't only learned about Spring AI, you have also learned how AI can be applied to interesting and challenging problems using Spring AI. By now you should feel comfortable using Spring AI to solve almost any problem you might face professionally or personally. Now your real journey has just begun. How will you use Spring AI in new and creative ways to build intelligent applications from your own ideas?

However, before we part, there is one more lesson.

- Learn to think critically and consciously about AI and its every application.

Therefore, we diverge from the low-level and intricate details of coding with Spring AI to zoom out and look at the bigger picture of how AI shapes and impacts humanity. What does it mean to be human? We all need to carefully consider our role and responsibilities when applying AI to shape human experience. In what follows, I share my perspective and thoughts about the past, present, and future state of software engineering in the brave new age of AI.

In order to go forward we must first look back.

From past to present

I have been fortunate in my lifetime and career to witness five major technological advancements.

The first seismic event to occur in my life was the personal computer in the early 1980s. This single event changed the trajectory of my life forever and is responsible for everything that has happened since.

I fondly remember back in September of 1983. I was nine years old. My parents had just bought me a Commodore 64 from Sears for $399 USD. It was my first personal com-

puter. Little did I know then just how much it would impact the rest of my life. Now look at me. I just wrote a book on Spring AI!

Next came the explosion of the internet back in the mid-1990s, which was quickly followed by widespread adoption and demand for broadband in early 2001. Broadband cemented the foundation for online e-commerce and was a pivotal shift from the early days of dialup. I can still hear the familiar sound of a "modem" ringing in my ears.

Then smartphones spearheaded the rise of mobile computing. Like the personal computer before it, the smartphone would forever change the way we consume information and interact with technology, as well as with each other.

The internet, smartphones, and eventually, the Internet of Things (IOT), paved the way for Cloud Computing. Suddenly everything was connected and online. While intelligence and real-time decisions remain at the edge, unlimited computing power can only be scaled and managed effectively in the cloud. It's no longer fiscally possible to keep pace with advancements in compute, especially as we approach the next era, quantum computing.

This brings me to my fifth and final major event in computing history, Artificial Intelligence. I firmly believe, like the personal computer, internet, cloud computing, and smartphone before it, AI will change the technology landscape forever. Not only that, it will be more disruptive than the previous four combined.

This timeline of events can be depicted in the following diagram:

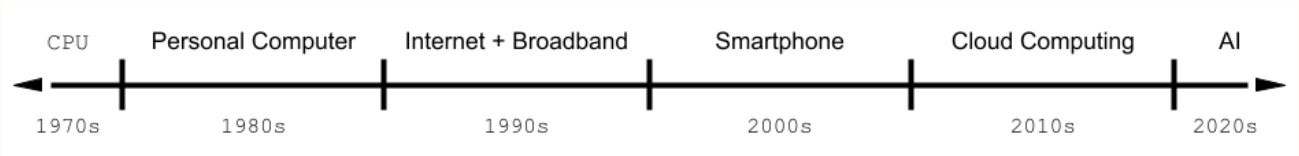


Figure 9.1: Short History of Computing

Let's zoom in on AI.

The disruptive force of AI

Let's get right to the question that everyone is wondering: "Will I lose my job? Is AI going to replace me?"

Inevitably, Yes! This is the nature of innovation and progress. This isn't the first time in history this has happened, though.

A history of disruption

We have consistently seen science and technology disrupt industries throughout history. It is what has allowed humankind to scale the production of nearly everything, from agriculture to medicine, the military, transportation, energy, communication, and even education. When you look back through time, it's not difficult to find disruptive events.

World War II produced the first electronic computer used to crack German codes once performed by code crackers. Human computers at NASA in the 1960s manually calculated flight trajectories and reentry for rockets carrying humans into space, of all things. Switchboard operators manually plugged cables into switchboards to connect calls at phone companies. Or, how about self-checkout and shopping on Amazon?

Whether we realize it or not, all people have had a hand in or are contributing to the very change they fear most.

Long gone are the days of full-service gas station attendants from the 1950s. They used to wash your windshields, check the engine oil and tire pressure, all while pumping gas for 15 cents a gallon. Eventually, gas stations will completely give way to electric vehicles.

By using smartphones, other devices, or any technology in general, you are impacting someone, somewhere, all the time. No longer using a thing has the same impact. Most of the time, we aren't thinking about it, but it's happening regardless. This kind of change occurs gradually over several decades, or sometimes, it only takes a few years.

Invention and Innovation through science and technology makes previous methods and tools obsolete, like the typewriter. Being a typists was a profession at one time. When I was in high school, we had to take a typing class. That was a skill needed at that time.

Now we use gestures to control our smart devices, speech to send text messages, and visuals or other biometrics to unlock our devices, homes, and cars. New methods are changing the interface and we are adapting. Typing, like handwriting, will be a lost skill, just like calligraphy before it. Typewriters will only be seen in museums.

Progression of change

What was once revered as a profession has become a (lost) art form only practiced by enthusiasts. Photography is one of the best examples. You only need to visit your local antique store to see all the things disrupted and discarded by innovation or invention.

The very nature of technology is to automate, to make things easier, to make them faster, to make them more reliable, and to reduce human error, especially for highly repetitive tasks.

For as long as humans stare up at the stars with childlike wonder and curiosity, we will imagine the impossible, we will build new, better tools, and as a result, disruption will happen. Disruption is the very fabric of change and progress.

You have a choice. You can either let yourself be changed by it or change with it. Either way, it is inevitable. After all, change is the only constant. However, it's not something we should fear. We should embrace it. We should all learn from it. AI is not only a powerful tool, but an incredible teacher. We can learn as much from AI as AI learns from us.

So, while some jobs will disappear, new opportunities requiring new skills will emerge from what was. This is not scary. This is exciting!

We as human beings have unparalleled creativity, intuition, the ability to think critically, learn, adapt, and evolve. A machine can't do that. Not yet. Maybe not ever.

We also have to remember that most technologies were not created to destroy life, even though they have great potential to do so, such as the Atomic Bomb. More often than not, the original intent was to improve life, such as Nuclear Energy.

A sure way to predict and understand the future is to create it yourself. Don't wait. Life is too short for all the fear, uncertainty, and doubt.

Besides, what do you really have to lose but time itself? Get started now!

Coding with AI

One of the hottest debates in technology at present is about generating code. You don't need to venture much further than LinkedIn to read all the crazy comments about how AI is devaluing the software engineering profession and will make software engineers obsolete. Or, how anyone can be, or already is, a software engineer, as if software engineering was all about "writing code." Writing code makes you a software engineer about as much as going to WebMD to find a treatment for an illness makes you a doctor. Like studying medicine in medical school, a background in computer science and a discipline in software, along with applied experience, will be more important than ever before. Having a solid foundation in the fundamentals is essential.

Code generation

Of course, AI wasn't the first tool that could generate code. Even within the Spring ecosystem, the Spring Roo project was a rapid application development (RAD) tool that helped Java developers use Spring to build Java-based enterprise software. It was the first Spring project to apply the convention-over-configuration principle by using templates and code generation. Of course, the use of this tool faded and eventually gave way to Spring Boot, using the same convention-over-configuration principle, but through auto-configuration rather than code generation.

AI can certainly generate a lot of code, really fast. But less code is more and speed only matters when you are going in the right direction to begin with. No matter though, if at first you don't succeed, try, try again. Just regenerate the code. It's really easy! It's cheap. It's like sitting at a high stakes poker table in a casino having never played poker before. In no time at all, you have exhausted your tokens and are down an easy grand. It's alright though, in several inflated areas, it is still cheaper per day than paying for a highly skilled software engineer. Plus, you don't have to pay the hidden costs of hiring an endless supply of expendable labor for much less in other countries, which breaks down once the standard of living exceeds ROI. The bottom line is, companies move on in order to keep shareholder margins constant.

It's the *sunk cost fallacy*, or making decisions based on past investments, also known as costs already incurred that cannot be recovered. I bet AI companies are already preparing. A whole new market emerges from non-coding coders, or more accurately, non-problem-solving problem solvers, who craft prompts all day until AI artificially spits out something marginally intelligent. Enter the prompt engineer. Consultants are already thinking there's good money to be made in prolonging this problem. Just wait until the tax bill arrives.

Beyond the code

Writing code was never the actual problem. It was about properly identifying the underlying problem. But most engineering efforts in the post-internet age have only been skin deep. For as long as I can remember, unvetted assumptions have been the Achilles' heel of the software industry. This is why in Agile software development it is important to get feedback, to refine your thinking, to refactor the code and iterate. It is a process of validating assumptions, simplifying, and continuous learning. Things change, understanding improves, and the path becomes a bit clearer. However, more often than not, many teams have deployed with much less by operating on costly assumptions.

Additionally, it's extremely important to manage scope. It's difficult to properly break a problem down and build just enough functionality to get feedback so that you haven't

sunk too far down in costs only to realize you were going in the wrong direction. It takes patience to learn that slow and deliberate shows you the way. Less code is more. The integration between hardware and software is essential. This is not "being agile"; this simply becomes common sense after you have grasped the fundamentals.

If you are embarrassed by your first product, then you are definitely moving too fast! You can pay that cost upfront or later. Either way, you will pay, and your reputation depends on choosing the right one.

What does everything I've said so far have to do with writing code using AI?

Again, it's worth repeating, writing code was never actually the problem. Sure, writing good code and sound logic is really difficult. There are a lot of details to manage and applying the right level of abstraction to build data structures and develop algorithms is essential. The more complex the problem, the more deliberate and organized you need to be. There is a reason why architecture and design are an art, because there is more than one way to solve a problem. This is true in software, in science, and even in math. In software, it's knowing how to apply art to the science.

Writing code and developing software are two very different things. Software development takes years of experience, patience, and a lot of practice to make good. Unfortunately, just good enough became the new standard when companies grew impatient and greedy. They weren't interested in solving the underlying issues surrounding software development—issues like stating the problem correctly, learning to manage scope and costs over time, adapting to change, and overcoming barriers to communication. They barely understood them, much less solved them. Why do you think we have so many software development processes to choose from?

This isn't anything new. AI is not going to come along and suddenly save us from ourselves by simply handling all the issues that have plagued the software industry for more than 50 years. This time around won't be different, either.

Electrolytes, it's what AI needs

Extreme delays, sunk budgets, overcommitting and underdelivering, suffering crippling downtime, drowning in complexity and eventually falling into oblivion. All these issues will persist despite the tremendous potential of AI. For one, the issues will persist because AI was trained on past data. Secondly, and more importantly, these issues will continue because we haven't got the organizational problem right. Hierarchical layers of bureaucracy and functional silos created unnecessary division of labor and significant over-

head. Another troubling fact is that there are many underqualified "leaders" with inflated egos making critical decisions that ultimately affect people.

None of us are innocent, though. Being that we are the now generation that wanted it yesterday for less, we thought we could go faster and get it done cheaper by treating software development like an assembly line. And, when I say "we", I mean management. Hordes of people with MBA degrees thought they could manage software without really understanding how software was built. Recall what happened in the dot-com era. For those readers who were not professional software engineers back then, I implore you to familiarize yourself with the thinking at that time. This mess stemmed from software engineers thinking they could manage a business while developing software and business people thinking writing code was synonymous with developing software.

Using AI, now all you need is an idea. You can come up with ideas and AI will simply generate the code for you. How difficult can it be? I mean, it's like building cars, right? Cars use software to regulate the engine and interface with the driver. Hmm, it must be the same thing! It just needs a fancy name. We can call it Lean software development. I will design the specs for a new flying car by mashing up a car with an airplane. Then, AI will use a 3D printer to build it. Finally, you can test it for me. How does that sound to you? Trust me, what could go wrong.

Lean, mean, fighting machine

One of the biggest issues in software development has been the division of labor. Basically, it boils down to taking away essential functions that were once the responsibility of software engineers. Even they have forgotten their responsibilities. So, new roles were created in hopes of streamlining the entire process to build software, like in an assembly line. This became known as Lean software development.

Two needless roles that came into existence in my experience were business analysts (BA) and quality assurance engineers (QA/QE). More technically capable business analysts were called business systems analysts (BSA). QA/QE engineers are known today as software developers in test (SDET). Business analysts, or business systems analysts, would analyze the problem, design a system to solve the problem, and then write use cases. The business analysts would pass their use cases off to the software engineers to write the code. Afterwards a QA team would test the software produced by the software engineers. The whole process was really "lean", in theory. Some got close and were better than others, but most were just terrible.

How can you come up with a "definition of done" without first understanding the problem? How do you know whether what you created works unless you experiment, test, and observe? Getting second-hand information is wasteful, and you can't claim the product is finished until it's proven to accomplish the task it was designed and built for correctly. Literally, the responsibility of a software engineer is to interface with the customer, or end-users, define the problem, understand the problem, experiment, and build prototypes. From the prototypes, you refine the code into working software. Then you test. Next, you get feedback. Finally, rinse, and repeat, otherwise known as iterate. This is Agile. This is common sense.

The irony of this Waterfall approach is that teams tried to claim they were "Agile" until the whole process broke down. This was largely due to communication problems born out of organizational silos and the whole one and done mindset. When the issues I mentioned above silently crept into projects, they tried to bandaid them by throwing more people at it. This only magnified the problem, so that's when management needed an excuse and declared that Agile doesn't scale. But it wasn't a resource problem; it was an organizational and utilization problem. You see, redundancy leads to a lack of ownership and self-worth. People become a cog in the machine that is easier to replace. At this juncture, complacency sets in, and people stop caring. The buck gets passed and this vicious cycle continues.

It's human nature to take simple things and make them more complicated than they really are. Also, we think certain things can't be that simple. If they were, then everyone would be doing it, and we can't sell that.

Only as strong as your weakest link

AI is a phenomenal tool that can really enhance the best performing software engineering teams out there with good principles and fundamentals, as well as magnify the problems produced by the worst teams out there. Think of it as a vector with both magnitude and direction. Whether you are going in the right direction or the wrong direction, it will magnify the effect.

Fortunately, there is already quite a lot of good information out there among the noise. And there are solid frameworks, like Spring AI, that can get you started in the right direction fairly quickly. Spring AI will greatly amplify your efforts because it has been built on a solid foundation with the Spring Framework. The Spring Framework was founded on solid and proven software principles that work and has been around for more than 20 years. Plus, if you combine the collective experience of the entire Spring team, it is very

difficult to match. Quite frankly, there is no substitute for experience because the only way you can earn that one is with time. Only time tells all.

It's important to remember that "writing code" is a variable effort with many facets, not the least of which is experience. Writing good code is an art form. And, developing reliable software that delights users is a process that takes much practice. Like any tool, AI can either make or break the outcome without the proper foundation.

The ethical dilemma

Gordon Moore, founder of Intel along with Robert Noyce, famously predicted that the number of transistors on a microchip would double every eighteen months. Now just look at the computing power we possess. We have orders of magnitude more power in the palms of our hands today than computers did only a few decades earlier.

The convergence of technologies, such as quantum computing and AI, will be the accelerant that drives the rate of change and new discoveries. Once the interface between humans and AI becomes symbiotic, the feedback loop will intensify our imaginations thus helping us formulate new ideas. As a result, there will be no limit to human potential or what will be possible in the future. The new universal speed limit won't be light, it will be thought. Thoughts, unlike light, have an amplifying effect. Sometimes our imaginations run wild.

Just because we can doesn't mean we should

The real question is not whether something will be possible, but whether we are ready for the impossible in a responsible and sustainable manner. At one time, none of what we are capable of today seemed possible hundreds or thousands of years ago. A little more than 100 years ago, in December of 1903, the Wright brothers managed to control and sustain flight. SpaceX has gone on to build rockets that land themselves. What will life be like 100 years from now? How about one thousand years from now?

In this moment we can only imagine, but by tomorrow, it will already become reality. It will be as common as the radio. There is hardly anyone alive today who did not grow up with a radio. Go back 100 years before the radio from the mid-1890s and it was probably pretty hard to imagine. The barrier has always been our ability to organize, connect, and process unfathomable amounts of data. Once the right tools are in place, the barrier will cease to exist, and the flood gates will open.

While studying computer science at university up until I graduated in the late 1990s, we were required to successfully complete a unit in computer ethics. I also recall reading an

article in Wired magazine around early 2000, interviewing Dr. Bill Joy, who co-founded Sun Microsystems. To paraphrase one of his quotes, "The last 20 years was about whether we could do something. The next 20 years will be about whether we should." This was precisely the point of our computer ethics study. Just because we can do something is not necessarily a free pass for, we should do that thing. We are capable of great things. However, there is a very fine line between great and terrible.

There is a lyric in Eddie Vedder's song, Alive, that says, "Do I deserve to be, and if so, if so, who answers, who answers". Indeed, this might be the most important question of them all. Who decides? I get that this particular lyric was not about whether a given technology should exist or not, but the philosophical principle is the same. In the future, the real division of power will be those with unlimited knowledge and the ability to control anything versus those without knowledge. There are already many people in government who don't understand how their world functions. As result, laws and regulations have succumbed to those with the power to control and manipulate.

Zero-sum game

At present, the world is shaped by two fundamental forces: countries with the biggest militaries and capitalism. The game has become to outlast the other and acquire the most resources, and this game is greatly aided by technological advances. Without a doubt, AI will be a force for both good and bad.

It is hard to grasp what the militaries of the world are fully capable of. After all, keeping your deepest secrets private is a strategic advantage. But technology is typically the deciding factor in whether one side has an insurmountable advantage over the other and is more often than not the ultimate factor to a successful outcome. You only need to observe the conflicts happening in the world to get a glimpse of the formidable power.

In a capitalistic society, 2 of the biggest industries to have significant consequential impact on the entire world are big oil and pharmaceuticals. In the early 2000s, big oil and industry stakeholders primarily lobbied to stifle the resurgence of commercial EV and hybrid vehicles. This type of self-preservation still has the most sizable environmental impact at 55%, contributing to all global energy-related CO_2 emissions. Today, while watching any program, it is drug commercial after drug commercial that promises to treat just about any medical condition imaginable. The side effects are worse than the condition the medication was meant to treat in many cases. They are not actually solving the underlying problem; they are just treating the symptoms. At present, the software industry is operating in much the same way.

The tech industry has created technology to enable bad actors the world over with zero liability. Online bullying through social media. Fake or stolen profiles on LinkedIn, Facebook and X. Scammers use WhatsApp to remain untraceable and conceal their identity from unsuspecting victims with no recourse. Technology companies have enabled rogue individuals and organizations at great scale. To make matters worse, these technology companies operate across international borders with very different laws and regulations. Unfortunately, legal jurisdiction is non-transferable, leaving many victims helpless. Legal teams attempting to combat the situation in over their heads and are ill-equipped to challenge the contemporary state of the union by applying outdated and no longer applicable laws. They lost before they ever sent their first court order.

Until AI isn't rooted in conquest and shaped by shareholder interests, which is no different than conflict of interest, the bottom line is that we can't be certain it won't be used to enrich the powerful and enslave the poor. In fact, if history has shown us anything, it most certainly will.

My only hope is that AI will be used to better humanity. As the technology evolves, because it will, and if AI ever reaches any form of sentience, that it can distinguish between good and bad intentions better than we can.

My call to action to you, the reader, is to pay attention, stay informed and challenge the thinking and actions of any AI provider when things are simply not right. When things are not aligned with our values. We all have a tremendous responsibility ahead of us and we must remain diligent if we are to sort out right from wrong.

Fortunately, from a technical perspective, Spring AI gives you the means to build proper checks and balances along the way. From Moderation to custom Advisors, and the use of RAG in order to ground the AI in reason, you are well positioned to build capable and positive human experiences that are truly intelligent.

What's next

There is much uncharted territory yet.

The possibilities are endless and no doubt will excite the imagination as well as induce fear, all at the same time. Future generations will need to contend with both moral and ethical decisions while determining what is only in the best interest for a select few.

From the smartphone to the smart home, intelligence will be seamlessly woven into the fabric of everything. I definitely think AI will play an ever-increasing role in real-time systems development where making accurate predictions and critical decisions at the

pivotal moment is key to success. AI combined with robotics will create automatons capable of serving as first responders, diffusing dangerous situations, or just assisting people with everyday tasks and companionship.

I also imagine a world where AI is blended with augmented and virtual reality to ensnare our senses. And, yes, I do imagine a world where a machine's ability to exhibit intelligent behavior will be indistinguishable from that of humans, thereby passing the Turing Test devised by Alan Turing way back in the 1950s. For now, it will remain a great tool to enhance our already existing skillsets and automate away repetition. Fortunately, we already have capable toolsets and frameworks, like Spring AI, to make the job of building intelligent and responsible applications easier.

Follow Spring AI at home on https://spring.io/projects/spring-ai#overview and on GitHub, https://github.com/spring-projects/spring-ai. Also follow what's happening in the Spring AI Community over at https://springaicommunity.mintlify.app/. Follow the AI providers: OpenAI, Anthropic, Cursor, Google, Meta, etc. Listen and watch carefully. What they say and do matters. This also shapes what library and framework teams focus on next and make accessible to developers.

Also, pay attention to other libraries and frameworks in the AI space, like **Embabel** and **Langchain4j**, as well as in other stacks, like Python. Use this book as a reference guide when starting any project. I have covered the APIs of Spring AI in detail and showed you how to use them to build practical examples. Most importantly, play, learn, grow, and have fun. So, what comes next? Personally, my money is on Agentic AI. But honestly, I don't know what will come next. No one does. I do know you should ask your software provider if AI is right for you. You shouldn't take AI if you experience a sudden increase in technical debt. Stop taking AI right away and consult your software professional if you experience increased outages or prolonged downtime, as this may be a sign of a severe or fatal condition called "complete system failure."

Don't get left behind. AI is the future. AI is now.

Assimilation is inevitable, "resistance is futile!" Sincerely, The Borg.

Live long and prosper my friends.

Get this book's PDF version and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require an invoice.

Previous chapter

‹ Chapter 8: Secret Agents

Next chapter

Chapter 10: Unlock Your Exclusive Benefits ›